



Red Hat OpenShift AI Self-Managed 3.4

Deploy models using Distributed Inference with llm-d

Deploy and serve large language models at scale in Red Hat OpenShift AI

Red Hat OpenShift AI Self-Managed 3.4 Deploy models using Distributed Inference with llm-d

Deploy and serve large language models at scale in Red Hat OpenShift AI

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As an administrator, you can use distributed inference to deploy and serve large language models at scale on Red Hat OpenShift AI.

Table of Contents

PREFACE	4
CHAPTER 1. DEPLOYING MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D	5
1.1. ENABLING DISTRIBUTED INFERENCE WITH LLM-D	5
CHAPTER 2. GATEWAY DISCOVERY FOR DISTRIBUTED INFERENCE WITH LLM-D	7
2.1. GATEWAY DISCOVERY FOR DISTRIBUTED INFERENCE WITH LLM-D	7
2.2. ENABLE GATEWAY DISCOVERY IN THE DASHBOARD	7
2.3. SELECT A GATEWAY FROM THE MODEL DEPLOYMENT WIZARD	8
2.4. SELECT GATEWAY USING YAML	10
CHAPTER 3. CONFIGURING AUTHENTICATION FOR DISTRIBUTED INFERENCE WITH LLM-D USING RED HAT CONNECTIVITY LINK	12
CHAPTER 4. ENABLING AUTHENTICATION AND AUTHORIZATION FOR AN LLM INFERENCE SERVICE	14
4.1. MAKING AUTHENTICATED INFERENCE REQUESTS TO DISTRIBUTED INFERENCE WITH LLM-D	15
CHAPTER 5. VLLM ARGUMENTS REFERENCE	18
5.1. COMMON USE CASES FOR VLLM ARGUMENTS	18
5.2. ARGUMENT REFERENCE	18
5.3. CONFIGURE VLLM ARGUMENTS FOR LLM-D DEPLOYMENTS	19
5.4. VLLM UVICORN ACCESS LOGS	21
5.5. ENABLE VLLM UVICORN ACCESS LOGS	22
5.6. EXAMPLE USAGE FOR DISTRIBUTED INFERENCE WITH LLM-D	24
5.6.1. Single-node GPU deployment	24
5.6.2. Multi-node deployment	24
5.6.3. Intelligent inference scheduler with KV cache routing	25
CHAPTER 6. CONFIGURING SCHEDULER SETTINGS FOR LLM INFERENCE SERVICES	26
6.1. CONFIGURE SCHEDULER SETTINGS FOR LLM INFERENCE SERVICES	26
6.1.1. Configuration methods	26
6.2. ENDPOINT PICKER ARCHITECTURE	26
6.2.1. Request flow through the Endpoint Picker	26
6.2.2. Complete request path	27
6.3. CONFIGURING SCHEDULER INLINE IN LLMINFERENCESERVICE	27
6.4. CONFIGURING SCHEDULER USING CONFIGMAP REFERENCES	28
6.5. SCHEDULER PLUGIN REFERENCE	30
6.5.1. Default plugins	30
CHAPTER 7. AUTOMATICALLY SCALE DISTRIBUTED INFERENCE WITH LLM-D MODEL DEPLOYMENTS	32
7.1. WORKLOAD VARIANT AUTOSCALER OVERVIEW	32
7.2. KEY METRICS FOR AUTOSCALING DECISIONS	33
7.3. HOW THE WORKLOAD VARIANT AUTOSCALER WORKS	33
7.3.1. Saturation scaling algorithm	34
7.4. ENABLE THE WORKLOAD VARIANT AUTOSCALER FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS	34
7.5. VARIANTAUTOSCALING CR CONFIGURATION REFERENCE	47
7.6. SATURATION SCALING CONFIGMAP REFERENCE	49
7.7. AUTOSCALING METRICS	50
7.8. HPA BEHAVIOR DEFAULTS	51
7.9. TROUBLESHOOTING THE WORKLOAD VARIANT AUTOSCALER	52
CHAPTER 8. MANAGING MIXED WORKLOADS WITH PRIORITY QUEUING	54

8.1. FLOW CONTROL AND PRIORITY-BASED QUEUING	54
8.2. INFERENCEOBJECTIVE CUSTOM RESOURCE REFERENCE	56
8.3. CONFIGURE FLOW CONTROL FOR DISTRIBUTED INFERENCE WITH LLM-D	57

PREFACE

As an administrator, you can use Distributed Inference with llm-d to deploy and serve large language models at scale on Red Hat OpenShift AI. You can configure distributed inference and set up authentication and authorization for your model deployments.

CHAPTER 1. DEPLOYING MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D

Distributed Inference with llm-d is a Kubernetes-native, open-source framework designed for serving large language models (LLMs) at scale. You can use Distributed Inference with llm-d to simplify the deployment of generative AI, focusing on high performance and cost-effectiveness across various hardware accelerators.

Key features of Distributed Inference with llm-d include:

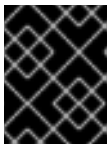
- Efficiently handles large models using optimizations such as prefix-cache aware routing and disaggregated serving.
- Integrates into a standard Kubernetes environment, where it leverages specialized components like the Envoy proxy to handle networking and routing, and high-performance libraries such as vLLM and NVIDIA Inference Transfer Library (NIXL).
- Tested recipes and well-known presets reduce the complexity of deploying inference at scale, so users can focus on building applications rather than managing infrastructure.

1.1. ENABLING DISTRIBUTED INFERENCE WITH LLM-D

This procedure describes how to create a custom resource (CR) for an **LLMInferenceService** resource. You replace the default **InferenceService** with the **LLMInferenceService**.

Prerequisites

- You have enabled the model serving platform.
- You have access to an OpenShift cluster running version 4.19.9 or later.
- OpenShift Service Mesh v2 is not installed in the cluster.
- Your cluster administrator has created a **GatewayClass** and a **Gateway** named **openshift-ai-inference** in the **openshift-ingress** namespace as described in [Gateway API with OpenShift Container Platform Networking](#).



IMPORTANT

Review the [Gateway API deployment topologies](#). Only use shared Gateways across trusted namespaces.

- Your cluster administrator has installed the **LeaderWorkerSet** Operator in OpenShift. For more information, see the [Leader Worker Set Operator documentation](#).
- If you are running OpenShift on a bare-metal cluster, your cluster administrator has an external entry point for the **openshift-ai-inference** Gateway service.



NOTE

By default, the Inference Gateway uses type: **LoadBalancer**. If the cluster does not already include support for **LoadBalancer** services, you can use the OpenShift option described in [Load balancing with MetalLB](#).

- You have enabled authentication as described in *Configuring authentication for Distributed Inference with llm-d*.

Procedure

1. Log in to the OpenShift console as a developer.
2. Create the **LLMInferenceService** CR with the following information:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: sample-llm-inference-service
spec:
  replicas: 2
  model:
    uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
    name: RedHatAI/Qwen3-8B-FP8-dynamic
  router:
    route: {}
    gateway: {}
    scheduler: {}
  template:
    containers:
      - name: main
    resources:
      limits:
        cpu: '4'
        memory: 32Gi
        nvidia.com/gpu: "1"
      requests:
        cpu: '2'
        memory: 16Gi
        nvidia.com/gpu: "1"
```

Customize the following parameters in the **spec** section of the inference service:

- **replicas** - Specify the number of replicas.
 - **model** - Specify the URI to the model based on how the model is stored (**uri**) and the model name to use in chat completion requests (**name**).
 - S3 bucket: **s3://<bucket-name>/<object-key>**
 - Persistent volume claim (PVC): **pvc://<claim-name>/<pvc-path>**
 - OCI container image: **oci://<registry_host>/<org_or_username>/<repository_name><tag_or_digest>**
 - HuggingFace: **hf://<model>/<optional-hash>**
 - **router** - Provide an HTTPRoute and gateway, or leave blank to automatically create one.
3. Save the file.

CHAPTER 2. GATEWAY DISCOVERY FOR DISTRIBUTED INFERENCE WITH LLM-D

Discover and select existing Kubernetes Gateway resources when deploying **LLMInferenceService** models. This Technology Preview feature builds on distributed inference capabilities by providing visibility into available Gateways.



IMPORTANT

Gateway discovery for Distributed Inference with llm-d is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

2.1. GATEWAY DISCOVERY FOR DISTRIBUTED INFERENCE WITH LLM-D

When deploying **LLMInferenceService** models, you can discover and select an existing Kubernetes Gateway resource directly from the model serving UI. This eliminates the need to manually configure a Gateway reference and provides visibility into which Gateway you have permission to use.

Gateway discovery is disabled by default and must be enabled by a cluster administrator through the dashboard configuration resource.

Which Gateways appear in the dropdown?

A Gateway is available only when both of the following conditions are met:

- RBAC permissions: You must have permissions to create **LLMInferenceService** resources in the target namespace.
- Listener configuration: The Gateway must include a listener configured to accept routes from the target namespace.

To deploy models with specific Gateway configurations or to reference multiple Gateways, use the YAML-based deployment approach. For more information, see [Select Gateway using YAML](#).



NOTE

The system does not validate against a specific **GatewayClass** resource or its controller. The default OpenShift Gateway controller provides all Distributed Inference with llm-d features. Third-party Gateway controllers might be incompatible with Distributed Inference with llm-d.

2.2. ENABLE GATEWAY DISCOVERY IN THE DASHBOARD

Enable Gateway discovery in the OpenShift AI dashboard so that you can discover and select Kubernetes Gateway resources during model deployment.

Prerequisites

- You have cluster administrator access to OpenShift.
- You have enabled distributed inference with the Gateway API installed. For more information, see [Enabling distributed inference](#).

Procedure

1. Enable Gateway discovery in the dashboard configuration:

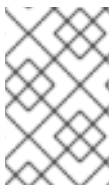
```
$ oc patch odhdashboardconfig odh-dashboard-config \
-n redhat-ods-applications \
--type merge \
-p '{"spec":{"dashboardConfig":{"llmGatewayField":true}}}'
```

Verification

1. Log in to the OpenShift AI dashboard.
2. In the navigation menu, click **AI Hub**.
3. Click a project name, then click **Deploy model**.
4. Verify that the **Gateway** field is displayed in **Advanced Settings**.

2.3. SELECT A GATEWAY FROM THE MODEL DEPLOYMENT WIZARD

You can discover and select an existing Kubernetes Gateway resource during an **LLMInferenceService** deployment through the OpenShift AI model deployment wizard. The Gateway can be the shared **openshift-ai-inference** Gateway configured during distributed inference setup or a namespace-scoped Gateway.



NOTE

If you edit a deployment with multiple Gateways through the model deployment wizard UI, the **Gateway Selection** field displays only the first Gateway. Gateway selection does not support MaaS Gateways.

Prerequisites

- You have enabled distributed inference as described in [Enabling distributed inference](#).
- Gateway discovery is enabled in the dashboard. For more information, see [Enabling Gateway discovery in the dashboard](#).
- You have permission to create **LLMInferenceService** resources in your namespace.
- At least one Gateway exists with a listener configured to accept routes from your namespace.
- You have a model stored in S3-compatible storage, a persistent volume claim (PVC), or an OCI container registry or accessible through a URI.

Procedure

1. Log in to the OpenShift AI dashboard.
2. In the navigation menu, click **AI Hub**.
3. Click the name of the project that you want to deploy a model in.
4. Click **Deploy model**.
5. Configure the model deployment properties as required for your model. For more information, see *Deploying models on the single-model serving platform* .
6. In **Advanced Settings**, locate the **Gateway Selection** section.
7. In the **Gateway** section, locate the **Select a gateway** dropdown.
8. Select the Gateway from the dropdown list.
9. Click **Next**.
10. Click **Deploy Model**.

Verification

1. On the **Models** page, verify that the **LLMInferenceService** is listed with a checkmark in the **Status** column.
2. Click the deployed model name to view details.
3. Verify that the Gateway reference is configured in the deployed model resource:

```
$ oc get llminferenceservices <llmisvc_name> -n <namespace> -o yaml
```

where:

<llmisvc_name>

Specifies the name of your **LLMInferenceService**.

<namespace>

Specifies the project namespace where you deployed the model.

The output includes the Gateway reference configuration. The Gateway reference appears under **spec.router.gateway**.

Troubleshooting

The Gateway dropdown is empty or does not show any available Gateways

- **Check RBAC permissions:** You might not have permission to create **LLMInferenceService** resources in your namespace. Contact your cluster administrator to verify your RBAC permissions.
- **Check listener configuration:** You might not have Gateways that are configured to accept routes from your namespace. Verify that at least one Gateway has a listener configured with **allowedRoutes.namespaces** that includes your namespace.

2.4. SELECT GATEWAY USING YAML

Select Kubernetes Gateway resources for an **LLMInferenceService** deployment by creating the resource using YAML. This approach enables you to reference multiple Gateways or specify detailed Gateway configurations that are not available through the model deployment wizard.



NOTE

If you edit a deployment with multiple Gateways through the model deployment wizard UI, the **Gateway Selection** field displays only the first Gateway. Gateway selection does not support MaaS Gateways.

Prerequisites

- You have enabled distributed inference as described in [Enabling distributed inference](#).
- You have permission to create **LLMInferenceService** resources in your namespace.
- At least one Gateway exists with a listener configured to accept routes from your namespace.
- You have a model stored in S3-compatible storage, a persistent volume claim (PVC), an OCI container registry, or accessible through a URI.

Procedure

1. Create a YAML file for your **LLMInferenceService** with Gateway references:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: <llmisvc_name>
  namespace: <namespace>
spec:
  modelSource:
    storageUri: <storage_uri>
  runtime:
    name: <runtime_name>
  router:
    gateway:
      refs:
        - name: <gateway_name>
          namespace: <gateway_namespace>
```

where:

<llmisvc_name>

Specifies the name for your **LLMInferenceService**.

<namespace>

Specifies your project namespace.

<storage_uri>

Specifies the location of your model, such as **s3://my-bucket/my-model/** or **pvc://my-pvc/my-model/**.

<runtime_name>

Specifies the serving runtime to use, such as **vllm-runtime**.

<gateway_name>

Specifies the name of the Gateway to use.

<gateway_namespace>

Specifies the namespace where the Gateway is located.

- To reference multiple Gateways, add additional entries to the **refs** list:

```
router:
  gateway:
    refs:
      - name: <gateway_name_1>
        namespace: <gateway_namespace_1>
      - name: <gateway_name_2>
        namespace: <gateway_namespace_2>
```

- Apply the YAML file:

```
$ oc apply -f <filename>.yaml
```

Verification

- Verify that the **LLMInferenceService** was created:

```
$ oc get llminferenceservices <llmisvc_name> -n <namespace>
```

where:

<llmisvc_name>

Specifies the name of your **LLMInferenceService**.

<namespace>

Specifies your project namespace.

- Verify the Gateway references in the deployed resource:

```
$ oc get llminferenceservices <llmisvc_name> -n <namespace> -o yaml
```

The output includes the Gateway reference configuration under **spec.router.gateway.refs**.

- Verify that the **LLMInferenceService** is ready:

```
$ oc get llminferenceservices <llmisvc_name> -n <namespace> -o
jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

The output should be **True** when the service is ready.

CHAPTER 3. CONFIGURING AUTHENTICATION FOR DISTRIBUTED INFERENCE WITH LLM-D USING RED HAT CONNECTIVITY LINK

Red Hat Connectivity Link provides Kubernetes-native authentication and authorization capabilities for Distributed Inference with llm-d inference endpoints. Connectivity Link works with the gateway to intercept incoming traffic before it reaches the vLLM inference service, validating the requests based on authentication tokens and authorization policies. For more information about Connectivity Link concepts and capabilities, see [Introduction to Connectivity Link](#).

Prerequisites

- You have installed Red Hat Connectivity Link version 1.1.1 or later. For more information, see [Installing Connectivity Link on OpenShift](#).
- You have access to the OpenShift CLI (**oc**).
- The ServiceAccount has permission to get the corresponding **LLMInferenceService** and you have generated a JSON web token (JWT).

Procedure

1. Create the Kuadrant custom resource (CR) to set up required objects:

```
oc apply -f - <<EOF
apiVersion: kuadrant.io/v1beta1
kind: Kuadrant
metadata:
  name: kuadrant
  namespace: kuadrant-system
EOF
```

2. Wait for Kuadrant to become ready:

```
oc wait Kuadrant -n kuadrant-system kuadrant --for=condition=Ready --timeout=10m
```

3. Add the **ServingCert** annotation to the Authorino Service:

```
oc annotate svc/authorino-authorino-authorization service.beta.openshift.io/serving-cert-secret-name=authorino-server-cert -n kuadrant-system
```

4. Wait for the secret to be created:

```
sleep 2
```

5. Update Authorino to enable SSL:

```
oc apply -f - <<EOF
apiVersion: operator.authorino.kuadrant.io/v1beta1
kind: Authorino
metadata:
  name: authorino
```



```

    namespace: kuadrant-system
spec:
  replicas: 1
  clusterWide: true
  listener:
    tls:
      enabled: true
      certSecretRef:
        name: authorino-server-cert
  oidcServer:
    tls:
      enabled: false
EOF

```

6. Verify that the Authorino pods are ready:

```

oc wait --for=condition=ready pod -l authorino-resource=authorino -n kuadrant-system --
timeout 150s

```

7. If OpenShift AI was installed before installing Connectivity Link and Kuadrant, restart the controllers:

```

oc delete pod -n redhat-ods-applications -l app=odh-model-controller
oc delete pod -n redhat-ods-applications -l control-plane=kserve-controller-manager

```

CHAPTER 4. ENABLING AUTHENTICATION AND AUTHORIZATION FOR AN LLM INFERENCE SERVICE

In OpenShift AI 3.0 and later, authentication and authorization are automatically enabled for **LLMInferenceService** resources when Red Hat Connectivity Link is configured. You can use the **security.opendatahub.io/enable-auth: "true"** annotation to explicitly enable authentication, such as re-enabling it after it was previously disabled.

Prerequisites

- You have configured Red Hat Connectivity Link for Distributed Inference with llm-d as described in *Configuring authentication for Distributed Inference with llm-d using Red Hat Connectivity Link*.
- You have created an **LLMInferenceService** resource as described in *Enabling Distributed Inference with llm-d*.
- You have access to the OpenShift CLI (**oc**).

Procedure

1. By default, authentication is enabled automatically. To explicitly enable authentication or to re-enable it after disabling, annotate your **LLMInferenceService** resource:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: sample-llm-inference-service
  annotations:
    security.opendatahub.io/enable-auth: "true"
spec:
  replicas: 2
  model:
    uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
    name: RedHatAI/Qwen3-8B-FP8-dynamic
  router:
    route: {}
    gateway: {}
    scheduler: {}
  template:
    containers:
      - name: main
        resources:
          limits:
            cpu: '4'
            memory: 32Gi
            nvidia.com/gpu: "1"
          requests:
            cpu: '2'
            memory: 16Gi
            nvidia.com/gpu: "1"
```

2. Apply the configuration:

```
oc apply -f <llm-inference-service-file>.yaml
```

Verification

- Confirm that the **LLMInferenceService** resource has the annotation:

```
oc get llminferenceservice sample-llm-inference-service -o
jsonpath='{.metadata.annotations.security\.opendatahub\.io/enable-auth}'
```

The command returns **true**.

- Verify that the inference service is protected by attempting to access it without authentication:

```
curl -v https://<inference-endpoint-url>/v1/models
```

The request returns a **401 Unauthorized** response, confirming that unauthenticated requests are rejected.

Additional resources

- [Override your Gateway policies for auth and rate limiting](#)

4.1. MAKING AUTHENTICATED INFERENCE REQUESTS TO DISTRIBUTED INFERENCE WITH LLM-D

When you enable authentication for an **LLMInferenceService**, you must include a valid JSON web token (JWT) in your inference requests. You can generate a token from a ServiceAccount or use an OIDC token from an identity provider.

Prerequisites

- You have enabled authentication for your **LLMInferenceService** as described in *Enabling authentication and authorization for an LLM inference service*.
- You have access to the OpenShift CLI (**oc**).
- You have the inference endpoint URL for your deployed model.

Procedure

- Create a ServiceAccount with permissions to access the **LLMInferenceService**:

```
oc create serviceaccount llm-user -n <namespace>
```

- Create a Role that grants permission to get the **LLMInferenceService**:

```
oc apply -f - <<EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: llm-inference-viewer
  namespace: <namespace>
```

```
rules:
- apiGroups: ["serving.kserve.io"]
  resources: ["llminferenceservices"]
  verbs: ["get"]
  resourceNames: ["<llm-inference-service-name>"]
EOF
```

3. Bind the Role to the ServiceAccount:

```
oc create rolebinding llm-user-binding \
  --role=llm-inference-viewer \
  --serviceaccount=<namespace>:llm-user \
  -n <namespace>
```

4. Generate a JWT token from the ServiceAccount:

```
TOKEN=$(oc create token llm-user -n <namespace> --duration=1h)
```

The **--duration** flag sets the token validity period. Adjust this value based on your workload requirements. For production use, consider using a shorter duration or integrating with an OIDC provider.

Verification

- Verify that the authenticated request was successful:

```
curl -v https://<inference-endpoint-url>/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${TOKEN}" \
  -d '{
    "model": "<model-name>",
    "messages": [{
      "role": "user",
      "content": "What is Red Hat OpenShift AI?"
    }]
  }'
```

A successful response indicates that authentication is working correctly.

- Verify that requests without authentication are rejected:

```
curl -v https://<inference-endpoint-url>/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "<model-name>",
    "messages": [{
      "role": "user",
      "content": "What is Red Hat OpenShift AI?"
    }]
  }'
```

The request returns a **401 Unauthorized** response with a message indicating that authentication is required.

- Verify that requests with an invalid or expired token are rejected:

```
curl -v https://<inference-endpoint-url>/v1/chat/completions \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer invalid-token" \  
-d '{  
  "model": "<model-name>",  
  "messages": [{  
    "role": "user",  
    "content": "What is Red Hat OpenShift AI?"  
  }]  
'
```

The request returns a **401 Unauthorized** response, confirming that invalid tokens are rejected.

Additional resources

- For information about OpenShift authentication, see [Understanding authentication](#).
- For information about ServiceAccount tokens, see [Using bound service account tokens](#).

CHAPTER 5. VLLM ARGUMENTS REFERENCE

When you deploy models using Distributed Inference with llm-d, the vLLM runtime handles inference requests. Red Hat AI Inference Server provides comprehensive documentation for vLLM runtime arguments and advanced configuration that you can apply to your llm-d deployments.

5.1. COMMON USE CASES FOR VLLM ARGUMENTS

Configure vLLM runtime arguments to achieve the following goals:

Optimize resource usage

- Reduce GPU memory consumption to fit larger models or enable more concurrent requests. For example, using **--gpu-memory-utilization=0.85** leaves headroom for system operations and prevents out-of-memory errors.
- Adjust context length limits to match your application's requirements. For example, use **--max-model-len=8192** for smaller context length to reduce memory requirements.

Improve performance

- Minimize latency in production by disabling verbose logging. For example, using **--disable-uvicorn-access-log** reduces I/O overhead from logging every HTTP request.
- Enable faster text generation using speculative decoding. For example, **--speculative-model=<draft-model>** and **--use-v2-block-manager** use a smaller draft model to predict tokens, which is verified by the main model.

Support specific model requirements

- Enable tool calling with custom models. For example, using **--enable-auto-tool-choice** and **-tool-call-parser=hermes** to enable the model to select tools automatically and parse tool calls using model-specific formats.
- Support custom chat templates for models that require specific formatting.
- Enable prefix caching to speed up requests with repeated prompts using **--enable-prefix-caching**.

Meet operational needs

- Configure trust settings for custom or private model sources using **--trust-remote-code**.
- Reduce log volume in production environments by disabling access logs.

5.2. ARGUMENT REFERENCE

Argument reference

- [vLLM Server Arguments](#)

Advanced features

- [Extending Red Hat AI Inference Server with Tool Calling Capabilities](#)
- [Using custom chat templates with models](#)

5.3. CONFIGURE VLLM ARGUMENTS FOR LLM-D DEPLOYMENTS

You can configure vLLM runtime arguments to optimize inference performance by using the standard Kubernetes container **args** field to control memory allocation, request handling, and model behavior without redeploying or rebuilding containers.

Prerequisites

- You have OpenShift AI 3.4 EA2 or later installed.
- You have deployed a model using Distributed Inference with llm-d for distributed inference or single-GPU deployments.
- You have stored a model in S3, a persistent volume claim (PVC), an OCI container registry, or HuggingFace.
- You have access to the OpenShift CLI (**oc**) or the OpenShift web console.

Procedure

1. Identify the vLLM arguments you want to configure. For the complete argument reference, see [Red Hat AI Inference Server vLLM Server Arguments](#).
2. Create or edit the **LLMInferenceService** custom resource to include the **args** field under **spec.template.containers** for the container named **main**.

Example vLLM argument configuration:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
  name: my-vllm-service
  namespace: <my_namespace>
spec:
  replicas: 2
  model:
    uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
    name: RedHatAI/Qwen3-8B-FP8-dynamic
  router:
    route: {}
    gateway: {}
    scheduler: {}
  template:
    containers:
      - name: main
        args:
          - --disable-uvicorn-access-log
          - --max-model-len=10000
        resources:
          limits:
            cpu: '4'
```

```
memory: 32Gi
nvidia.com/gpu: "1"
requests:
  cpu: '2'
  memory: 16Gi
  nvidia.com/gpu: "1"
```

where:

- **--disable-uvicorn-access-log**: Disables HTTP access logs to reduce log volume.
- **--max-model-len=10000**: Sets the maximum context length to 10,000 tokens.



NOTE

The arguments that you specify are merged with the default arguments provided by the system. If you specify the same argument as a default, your value takes precedence. This allows you to override specific defaults without affecting other default arguments.

3. Apply the custom resource.

```
$ oc apply -f -n <my_namespace> llminferenceservice.yaml
```

Example output

```
llminferenceservice.serving.kserve.io/my-vllm-service configured
```

Verification

1. Verify that the arguments are applied correctly.
 - a. Check that the **LLMInferenceService** is running:

```
$ oc get llminferenceservice my-vllm-service -n <my_namespace>
```

Example output

```
NAME           READY  AGE
my-vllm-service  True   5m
```

The READY column shows **True**.

- b. Verify your custom arguments appear in the pod specification:
Get the pod name:

```
$ oc get pods -n <my_namespace> -l app.kubernetes.io/name=my-vllm-service
```

Inspect the container arguments:

```
$ oc describe pod <pod_name> -n <my_namespace> | grep -A 5 "Args:"
```


Example output

```
Args:
  --disable-uvicorn-access-log
  --max-model-len=10000
```

Your custom arguments should appear in this list.

- c. Test that inference requests work with your configured arguments:
Get the route URL:

```
$ oc get llm_svc -n <my_namespace> my-vllm-service -o jsonpath='{.status.url}'
```

Send a test request:

```
$ curl -X POST https://<route_url>/v1/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $(oc whoami -t)" \
-d { "model": "RedHatAI/Qwen3-8B-FP8-dynamic", "prompt": "Explain what Red Hat OpenShift AI is in one sentence.", "max_tokens": 100 }
```

A successful response confirms that your vLLM service is running with the configured arguments.

Additional resources

- [Red Hat AI Inference Server vLLM Server Arguments](#)
- [Using custom chat templates with models](#)

5.4. VLLM UVICORN ACCESS LOGS

You can control vLLM uvicorn access log behavior based on your debugging and monitoring needs. In Red Hat OpenShift AI 3.4, uvicorn access logs are disabled by default for vLLM deployments that use Distributed Inference with llm-d. This change prevents infrastructure overload caused by high-frequency health checks.

The default **LLMInferenceServiceConfig** template hardcodes the **--disable-uvicorn-access-log** flag in the vLLM startup command. Because this flag is embedded in the container startup command rather than the args list, re-enabling access logs requires overriding the entire command field.



NOTE

Disabling uvicorn access logs does not affect metrics collection. The **/metrics** endpoint continues to function normally, and Prometheus can still scrape metrics regardless of access log configuration. Metrics provide aggregated performance data, while access logs provide request-level details.

When to re-enable access logs

You might need to re-enable uvicorn access logs in the following scenarios:

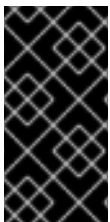
- Debugging API usage patterns to understand which endpoints are being called and how frequently
- Monitoring traffic patterns for capacity planning or usage analysis
- Troubleshooting request and response issues that are not visible in metrics alone
- Investigating unexpected API behavior or error conditions
- Analyzing response times and latency patterns across different endpoints

When you enable access logs, you must balance the need for visibility with the infrastructure impact. Consider enabling logs temporarily during active debugging sessions and disabling them again when troubleshooting is complete.

For vLLM 0.16 or later, endpoint-specific log filtering provides a better alternative that maintains visibility into actual API usage while filtering noisy health check logs.

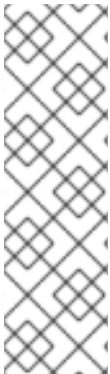
5.5. ENABLE VLLM UVICORN ACCESS LOGS

You can enable vLLM uvicorn access logs to debug API usage patterns and troubleshoot request issues. Enabling access logs generates significant log volume that can overwhelm the OpenShift web console and log aggregation infrastructure. vLLM uvicorn access logs are disabled by default.



IMPORTANT

Re-enabling uvicorn access logs requires overriding the container startup command because the **--disable-uvicorn-access-log** flag is hardcoded in the default command, not in the args list. Overriding the command field is an advanced configuration that replaces the entire default startup behavior.



NOTE

The default **LLMInferenceServiceConfig** template includes a RoCE auto-detection prelude in the **command** field for high-performance multi-node deployments. The minimal override shown in this procedure drops that prelude, which is appropriate for the typical access-log debugging audience with single-node deployments. If your deployment relies on RoCE auto-detection, copy the full **command** block from the upstream **LLMInferenceServiceConfig** template at [opendatahub-io/kserve config-llm-template.yaml](https://opendatahub-io.github.io/kserve/config-llm-template.yaml) and remove only the line containing **--disable-uvicorn-access-log** rather than using the minimal override below.

Prerequisites

- A **LLMInferenceService** CR instance
- You have access to the OpenShift CLI (**oc**) or the OpenShift web console
- You understand that enabling access logs generates significant log volume due to frequent health check and metrics polling (approximately every 200 milliseconds)

Procedure

1. Edit your **LLMInferenceService** custom resource to override the container command.

The following example shows how to override the startup command to exclude the **--disable-uvicorn-access-log** flag:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: my-vllm-service
  namespace: <namespace>
spec:
  replicas: 2
  model:
    uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
    name: RedHatAI/Qwen3-8B-FP8-dynamic
  router:
    route: {}
    gateway: {}
    scheduler: {}
  template:
    containers:
      - name: main
        command:
          - /bin/bash
          - -c
          - |
            vllm serve "$@"
        --
    args:
      - /mnt/models
      - --served-model-name=RedHatAI/Qwen3-8B-FP8-dynamic
      - --port=8000
      - --max-model-len=10000
      - --gpu-memory-utilization=0.9
      # Remove the following three lines if your deployment does not use TLS
      - --enable-ssl-refresh
      - --ssl-certfile=/var/run/kserve/tls/tls.crt
      - --ssl-keyfile=/var/run/kserve/tls/tls.key
    resources:
      limits:
        cpu: '4'
        memory: 32Gi
        nvidia.com/gpu: "1"
      requests:
        cpu: '2'
        memory: 16Gi
        nvidia.com/gpu: "1"
```

where:

- **command:** Overrides the default startup command with a minimal bash wrapper that calls **vllm serve** and forwards the contents of **args** to it as positional arguments via **"\$@"**. The **--** separator after the script body is required for bash to treat subsequent entries as positional parameters rather than bash options.
- **args:** Contains the vLLM server arguments. The first entry, **/mnt/models**, is the required model path that the storage initializer mounts into the container. The remaining entries are vLLM CLI flags. Replace the **--served-model-name** value with the value of

spec.model.name from your **LLMInferenceService**.



NOTE

When you override the **command** field, you replace all default command-level behavior, including the **--disable-uvicorn-access-log** flag that is normally applied. By omitting this flag in your custom command, uvicorn access logs are enabled.

2. Apply the custom resource:

```
$ oc apply -f llminferenceservice.yaml -n <namespace>
```

Verification

1. Verify that the LLMInferenceService is running:

```
$ oc get llminferenceservice <service_name> -n <namespace>
```

The output shows the service status:

```
NAME          READY  AGE
my-vllm-service  True   5m
```

2. View the pod logs to confirm that access log entries appear:

```
$ oc logs <pod_name> -n <namespace> | grep "HTTP/1.1"
```

where **<pod_name>** is the name of one of the vLLM pods.

The output shows HTTP access log entries:

```
INFO: 172.30.45.2:54321 - "GET /metrics HTTP/1.1" 200 OK
INFO: 172.30.45.2:54322 - "GET /health HTTP/1.1" 200 OK
INFO: 172.30.45.2:54323 - "POST /v1/completions HTTP/1.1" 200 OK
```

Verify the log format includes the HTTP method, endpoint path, and status code.

5.6. EXAMPLE USAGE FOR DISTRIBUTED INFERENCE WITH LLM-D

These examples show how to use Distributed Inference with llm-d in common scenarios.

5.6.1. Single-node GPU deployment

Use single-GPU-per-replica deployment patterns for development, testing, or production deployments of smaller models, such as 7-billion-parameter models.

For examples using single-node GPU deployments, see [Single-Node GPU Deployment Examples](#).

5.6.2. Multi-node deployment

For examples using multi-node deployments, see [DeepSeek-R1 Multi-Node Deployment Examples](#).

5.6.3. Intelligent inference scheduler with KV cache routing

You can configure the scheduler to track key-value (KV) cache blocks across inference endpoints and route requests to the endpoint with the highest cache hit rate. This configuration improves throughput and reduces latency by maximizing cache reuse.

For an example, see [Precise Prefix KV Cache Routing](#).

CHAPTER 6. CONFIGURING SCHEDULER SETTINGS FOR LLM INFERENCE SERVICES

You can configure custom scheduler settings for large language model (LLM) inference services to optimize request routing and improve performance characteristics such as cache efficiency and load distribution.

6.1. CONFIGURE SCHEDULER SETTINGS FOR LLM INFERENCE SERVICES

You can configure custom scheduler settings for large language model (LLM) inference services to optimize request routing and improve performance characteristics, such as prefix cache hit rates and request distribution patterns.

The EndpointPicker determines how incoming requests are routed to available model endpoints in your LLM inference service deployment. When multiple replicas of a model are running, the scheduler decides which specific replica handles each request.

If you do not specify custom scheduler configuration, the system uses a default profile that balances requests across replicas by combining queue depth, for example with a **weight** value of **2**, and a prefix cache hit scoring, for example with a **weight** value of **3**, then selects the highest-scoring replica. This default prioritizes KV cache reuse over even load distribution to minimize redundant computation.

6.1.1. Configuration methods

You can configure scheduler settings using two methods:

Inline configuration

Inline configuration embeds the scheduler settings directly in the **LLMInferenceService** resource specification. This method is suitable for simple configurations or when you have unique scheduler settings for a specific service. It keeps all settings for a service in one place for easy viewing.

ConfigMap-based configuration

ConfigMap-based configuration stores scheduler settings in a Kubernetes ConfigMap and references it from the **LLMInferenceService** resource. This method is ideal when you want to reuse the same scheduler configuration across multiple LLM inference services, or when you prefer to manage configurations separately from service definitions. It also enables centralized configuration management and easier updates to multiple services.

The two configuration methods are mutually exclusive. You cannot use both inline and ConfigMap-based configuration in the same **LLMInferenceService** resource.

6.2. ENDPOINT PICKER ARCHITECTURE

The Endpoint Picker (EPP) is the request orchestration component in Distributed Inference with llm-d that sits between the Inference Gateway and backend model servers. The Endpoint Picker contains two distinct layers that control how requests flow through the system: the flow control layer and the Scheduling layer.

6.2.1. Request flow through the Endpoint Picker

When an inference request arrives at the Inference Gateway, it passes through the Endpoint Picker before reaching a backend model server. The Endpoint Picker processes each request through two sequential layers:

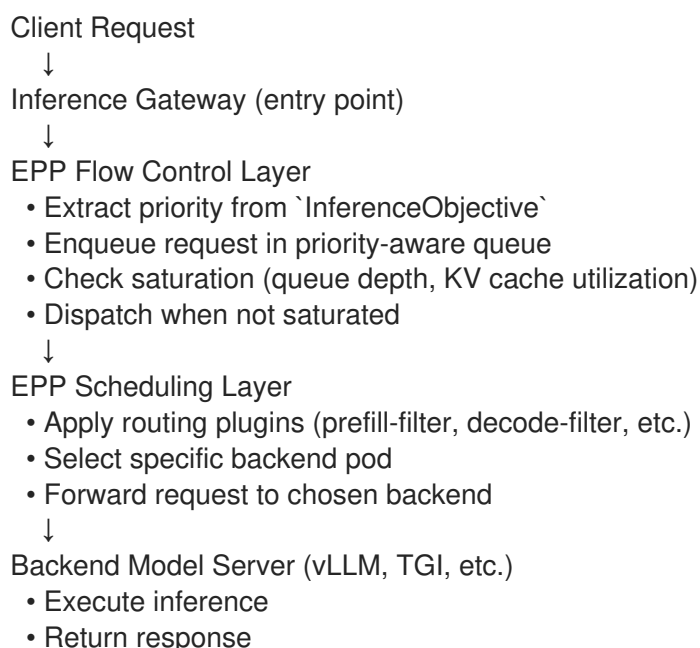
Flow Control layer

Controls **when** and **in what order** requests are dispatched to backends. This layer manages priority-based queuing, saturation detection, and load shedding to enforce service-level objectives and prevent pool overload.

Scheduling layer

Controls **where** each request is routed among available backends. This layer uses routing plugins to select the optimal backend pod based on factors such as prefill versus decode capacity, prefix cache affinity, and load distribution.

6.2.2. Complete request path



The Scheduling layer is always enabled. Flow Control is optional and can be enabled through the **flowControl** feature gate for priority-based queuing and saturation detection.

6.3. CONFIGURING SCHEDULER INLINE IN LLMINFERENCESERVICE

You can configure scheduler settings directly within your `LLMInferenceService` resource specification by using inline configuration. This approach embeds the scheduler configuration in the same YAML file as your service definition, making it suitable for service-specific settings.

Prerequisites

- You have cluster administrator privileges or namespace administrator privileges for the namespace where you want to deploy the LLM inference service.
- You have installed OpenShift AI and configured model serving.
- You have a model that you want to deploy as an LLM inference service, or you have an existing **LLMInferenceService** resource that you want to update.

Procedure

1. Create a YAML file for your **LLMInferenceService** resource or open an existing service definition for editing.
2. Add the **router.scheduler.config.inline** section to the service specification:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: llm-service-with-scheduler # the namespace where you want to deploy the service
  namespace: <your_namespace>
spec:
  model:
    uri: hf://<your_model_uri>
    name: <your_model_name>
  router:
    scheduler:
      config:
        inline:
          apiVersion: inference.networking.x-k8s.io/v1alpha1
          kind: EndpointPickerConfig
          plugins:
            - type: prefix-cache-scorer
            - type: queue-scorer
          schedulingProfiles:
            - name: default
            plugins:
              - pluginRef: queue-scorer
                weight: 2
              - pluginRef: prefix-cache-scorer
                weight: 3
```

- **router.scheduler.config.inline.plugins.type:** This example enables prefix cache scoring and queue scoring to route similar prompts to the same replica. You can configure the scheduler plugins according to your requirements.

3. Apply the configuration to your cluster:

```
$ oc apply -f llm-service.yaml
```

4. Verify that the LLM inference service is running with the custom scheduler configuration:

```
$ oc get llminferenceservice llm-service-optimized -n model-serving
```

The output shows the service status. Wait until the service reaches the **Ready** state.

Verification

- Send test requests to your LLM inference service to verify that it is handling requests correctly with the configured scheduler settings.

6.4. CONFIGURING SCHEDULER USING CONFIGMAP REFERENCES

You can configure scheduler settings by storing the configuration in a Kubernetes ConfigMap and referencing it from your LLMInferenceService resource. This approach is ideal for sharing configurations across multiple services or managing configurations separately from service definitions.

Prerequisites

- You have cluster administrator privileges or namespace administrator privileges for the namespace where you want to deploy the LLM inference service.
- You have installed OpenShift AI and configured model serving.
- You have a model that you want to deploy as an LLM inference service, or you have an existing **LLMInferenceService** resource that you want to update.
- You are familiar with Kubernetes ConfigMaps.

Procedure

1. Create a ConfigMap that contains your scheduler configuration. Create a YAML file named **scheduler-config.yaml** with the following content:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: llm-scheduler-config
  namespace: <your_namespace> # Replace with your namespace
data:
  default-scheduler: |
    apiVersion: serving.kserve.io/v1alpha1
    kind: EndpointPickerConfig
    plugins:
      - type: prefix-cache-scorer
      - type: queue-scorer
    schedulingProfiles:
      - name: default
        plugins:
          - pluginRef: queue-scorer
            weight: 2
          - pluginRef: prefix-cache-scorer
            weight: 3
```

- **data:** This section contains a key, **default-scheduler**, with the scheduler configuration as its value.
 - **data.default-scheduler:** The scheduler configuration. You can include multiple configurations by adding more keys to the **data** section.
2. Apply the ConfigMap to your cluster:

```
$ oc apply -f scheduler-config.yaml
```

3. Verify that the ConfigMap was created successfully:

```
$ oc get configmap llm-scheduler-config -n <your_namespace>
```

4. Create or edit your **LLMInferenceService** resource to reference the ConfigMap. For example, create a YAML file named **llm-service.yaml**:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: llm-service-with-configmap
  namespace: <your_namespace>
spec:
  model:
    uri: hf://<your_model_uri>
    name: <your_model_name>
  router:
    scheduler:
      config:
        ref:
          name: llm-scheduler-config
          key: default-scheduler
```

- **ref.name**: Specifies the ConfigMap name
 - **ref.key**: Specifies which key within the ConfigMap contains the scheduler configuration.
5. Apply the LLMInferenceService configuration:

```
$ oc apply -f llm-service.yaml
```

6. Verify that the LLM inference service is running with the scheduler configuration from the ConfigMap:

```
$ oc get llminferenceservice llm-service-with-configmap -n <your-namespace>
```

Wait until the service reaches the **Ready** state.

Verification

- Send test requests to your LLM inference service to verify that it handles requests correctly with the configured scheduler settings.

6.5. SCHEDULER PLUGIN REFERENCE

Scheduler plugins implement specific routing logic that determines how the EndpointPicker distributes requests across model replicas. You can configure these plugins in the **EndpointPickerConfig** to customize request routing behavior.

When multiple plugins are configured, the scheduler combines their scores to make the final routing decision. The scheduler evaluates each plugin for every request and selects the endpoint with the highest combined score.

6.5.1. Default plugins

queue-scorer

The **queue-scorer** plugin scores replicas based on queue depth. Replicas with shorter queues score higher, helping to distribute requests evenly across available replicas and prevent overloading individual instances.

prefix-cache-scorer

The **prefix-cache-scorer** plugin optimizes request routing for improved prefix cache hit rates. It routes requests with similar prompt prefixes to the same model replica, increasing the likelihood that the replica can reuse previously computed results for common prompt prefixes.

CHAPTER 7. AUTOMATICALLY SCALE DISTRIBUTED INFERENCE WITH LLM-D MODEL DEPLOYMENTS

You can configure intelligent, inference-aware automatic scaling for Distributed Inference with llm-d model deployments using the workload variant autoscaler (WVA). WVA automatically adjusts replica counts and AI accelerator allocation based on real-time traffic signals and hardware capacity, helping you optimize infrastructure usage while maintaining inference performance.



IMPORTANT

Workload variant autoscaler is a Developer Preview feature only. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to upcoming product features in advance of their possible inclusion in a Red Hat product offering, enabling customers to test functionality and provide feedback during the development process. These features might not have any documentation, are subject to change or removal at any time, and testing is limited. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.

7.1. WORKLOAD VARIANT AUTOSCALER OVERVIEW

The workload variant autoscaler (WVA) is a Kubernetes controller that provides intelligent, inference-aware autoscaling for Distributed Inference with llm-d model deployments. You can use the WVA to automatically adjust replica counts and AI accelerator allocation based on real-time traffic signals, hardware capacity, and workload characteristics instead of relying on generic metrics such as CPU or memory utilization.

Traditional Kubernetes autoscaling approaches rely on metrics such as CPU utilization, memory consumption, or queries per second (QPS). These metrics do not accurately represent the resource demands of large language model (LLM) inference workloads because LLM inference is GPU-bound, not CPU-bound. CPU utilization does not reflect the actual load on the inference server.

The WVA addresses these limitations by deriving scaling decisions from inference-specific signals that directly reflect the state of the model server.

The WVA operates as part of the Distributed Inference with llm-d inference stack alongside the following components:

vLLM model servers

The inference serving backends that the WVA scales.

Inference Gateway

Routes incoming requests to available model server replicas.

Inference scheduler

Selects optimal endpoints for each request based on load and locality.

KV cache management

Manages key-value cache storage and transfer between replicas. This component is embedded in the inference scheduler and does not run as a separate container.

KEDA

Provides the autoscaling actuator that the WVA uses to manage HPA resources. KEDA reads the **wva_desired_replicas** metric from Prometheus and manages the HPA resource that scales the target deployment.

7.2. KEY METRICS FOR AUTOSCALING DECISIONS

The workload variant autoscaler (WVA) uses the following inference-specific metrics from vLLM model servers to drive scaling decisions:

- **KV cache utilization (`kserve_vllm:kv_cache_usage_perc`):** The percentage of key-value cache in use on each replica. The WVA queries the maximum value over a 1-minute window. A replica is considered saturated when its KV cache utilization reaches the configured **kvCacheThreshold**.
- **Queue length (`kserve_vllm:num_requests_waiting`):** The number of requests waiting to be processed on each replica. The WVA queries the maximum value over a 1-minute window. A replica is considered saturated when its queue length reaches the configured **queueLengthThreshold**.

The WVA evaluates these metrics across all replicas of a deployment to calculate spare capacity and determine whether to scale up or scale down.

7.3. HOW THE WORKLOAD VARIANT AUTOSCALER WORKS

The workload variant autoscaler (WVA) is deployed as a controller that monitors **VariantAutoscaling** custom resources and adjusts replica counts for model server deployments. To deploy the WVA controller, you must enable it in the **DataScienceCluster** custom resource.

There is a single inference stack per namespace. You cannot deploy multiple inference stacks in the same namespace.

You can create multiple **LLMInferenceService** CRs within a single stack to serve the same model on different accelerator types. For example, you might have one deployment on H100 GPUs and another on A100 GPUs. The WVA coordinates scaling across all variants that share the same model identifier, scaling up the cheapest variant first.

The WVA uses a saturation-based spare capacity model to make scaling decisions. It evaluates three main dimensions:

Token-level demand estimation

By measuring KV cache utilization as a proxy for tokens actively being processed, and the number of requests waiting in queue, the WVA computes token-level demand rather than relying on generic throughput metrics such as CPU or memory utilization.

Saturation-aware spare capacity

The WVA compares token-level demand against each replica's effective capacity: the minimum of its memory-bound KV cache size limit and compute-bound limit, or batch saturation. Scaling decisions are triggered when spare capacity across replicas falls below a threshold, rather than waiting for replicas to become fully saturated.

Hardware-aware optimization

At a per-variant level, the WVA accounts for AI accelerator saturation levels. In the **LLMISVC** CRD, different accelerators can be selected for the requirements of each variant, for example:

- Prefill optimized

- Generic inference workers capable of both workloads
- Other configurations that can change the model server's behavior.

7.3.1. Saturation scaling algorithm

The workload variant autoscaler (WVA) uses a saturation-based spare capacity model to make scaling decisions. The algorithm evaluates whether the current replica set has sufficient spare capacity to absorb traffic increases.

The scaling algorithm works as follows:

1. **Identify non-saturated replicas** A replica is non-saturated if its KV cache utilization is below **kvCacheThreshold** and its queue length is below **queueLengthThreshold**.
2. **Calculate spare capacity**: For each non-saturated replica, the WVA calculates the spare KV capacity and spare queue capacity.
3. **Average spare capacity**: The WVA averages the spare capacity across all non-saturated replicas.
4. **Scale-up decision**: The WVA signals a scale-up if the average spare KV capacity is below **kvSpareTrigger** or the average spare queue capacity is below **queueSpareTrigger**.
5. **Scale-down safety check**: The WVA simulates removing one replica and redistributing load. Scale-down is safe only if remaining spare capacity still exceeds the triggers and at least 2 non-saturated replicas exist.

A variant is eligible for scaling if it does not already have a pending scaling decision from a previous reconciliation cycle. If a variant's desired replica count differs from its current count, the WVA preserves the existing decision and skips that variant when selecting candidates for new scaling actions.

To prevent over-provisioning during the model loading period, the WVA skips variants that have pending replicas when selecting a variant to scale up. A replica is considered pending when the pod exists but is not yet reporting metrics. This cascade scaling prevention ensures that only variants with all replicas ready are eligible for additional scale-up.

7.4. ENABLE THE WORKLOAD VARIANT AUTOSCALER FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS

You can enable intelligent autoscaling for your Distributed Inference with llm-d model deployments by configuring the workload variant autoscaler (WVA). The WVA controller is automatically deployed when the OpenShift AI Operator is installed. After you configure autoscaling in the **LLMInferenceService** custom resource, the WVA automatically adjusts the replica count of your model server based on real-time inference traffic and AI accelerator capacity.

Prerequisites

- You have an OpenShift cluster on version **4.20** or later.
- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.

- You have installed Red Hat OpenShift AI 3.4 with the Distributed Inference with llm-d stack enabled.



IMPORTANT

Dependencies for installing and using the Distributed Inference with llm-d stack require configuration of the underlying Red Hat OpenShift AI deployment. You cannot use WVA without first configuring OpenShift AI.

- Optional: You have installed **jq**.
- You have installed compatible AI accelerators in the cluster.
- You have enabled OpenShift User Workload Monitoring (UWM) in the cluster for Prometheus metrics collection. See [Configuring user workload monitoring](#).
- You have installed the **custom-metrics-autoscaler** Operator from OperatorHub. See [Automatically scaling pods with the Custom Metrics Autoscaler Operator](#).
- You have installed the **Red Hat Connectivity Link** Operator from OperatorHub. See [Installing Red Hat Connectivity Link](#).
- You have installed the **Red Hat OpenShift Service Mesh 3** Operator in your cluster.



NOTE

The Operator is installed by default on any OpenShift cluster version 4.20 or later.

- **DataScienceClusterInitialization (DSCI)** and **DataScienceCluster (DSC)** CRDs exist in your cluster, enabling the **workload-variant-autoscaler-controller-manager**, **llmisvc-controller-manager** and **kserve-controller-manager** controllers. The **DataScienceClusterInitialization** controller is automatically created by the OpenShift AI Operator. This example excerpt from the **DataScienceCluster** manifest enables the WVA controller:

```
apiVersion: datasciencecluster.opendatahub.io/v2
kind: DataScienceCluster
metadata:
  name: default-dsc
  labels:
    app.kubernetes.io/name: datasciencecluster
spec:
  components:
    kserve:
      # Create the KServe and LLMISVC controller managers
      managementState: "Managed"
    nim:
      managementState: "Managed"
      rawDeploymentServiceConfig: "Headed"
    wva:
      # Create the workload-variant-autoscaler controller manager
      managementState: "Managed"
      # ...Other DSC components as desired
```

- You have confirmed that no other **LLMInferenceService** CR exists in the namespace you intend to deploy the WVA in. Each namespace should contain a single Distributed Inference with llm-d inference stack only.

Procedure

- Verify that the required controllers exist in the cluster. Run the following command:

```
$ oc get pods -n redhat-ods-applications \
  -l 'app.kubernetes.io/name in (kserve-controller-manager, llmsvc-controller-manager,
  workload-variant-autoscaler)'
```

All controllers should report as ready:

NAME	READY	STATUS	RESTARTS	AGE
kserve-controller-manager	1/1	Running	0	38m
llmsvc-controller-manager	1/1	Running	0	38m
workload-variant-autoscaler-controller-manager	1/1	Running	0	38m

- Verify the default scaling configuration.
Run the following command to inspect the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap:

```
$ oc get cm workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications
-o jsonpath='{.data.default}'
```

You should see the following default values:

```
kvCacheThreshold: 0.80
queueLengthThreshold: 5
kvSpareTrigger: 0.1
queueSpareTrigger: 3
enableLimiter: false
```

These values represent the default scaling thresholds that the WVA uses to determine when to scale your inference workloads up or down.

- Grant the KEDA controller permissions to read from OpenShift monitoring resources.



NOTE

The Custom Metrics Autoscaler Operator does not automatically have permission to read metrics from the **openshift-monitoring** or **openshift-user-workload-monitoring** stack because OpenShift restricts access to cluster monitoring APIs. Access to Prometheus and Thanos endpoints requires explicit RBAC permissions such as the **cluster-monitoring-view** role.

- Authorize KEDA to read OpenShift user workload monitoring (UWM) metrics. Create the following CRs:

```
apiVersion: v1
kind: ServiceAccount
metadata:
```



```

name: keda-metrics-reader
namespace: openshift-keda
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: keda-metrics-reader-monitoring
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-monitoring-view
subjects:
- kind: ServiceAccount
  name: keda-metrics-reader
  namespace: openshift-keda
---
apiVersion: v1
kind: Secret
metadata:
  name: keda-metrics-reader-token
  namespace: openshift-keda
  annotations:
    kubernetes.io/service-account.name: keda-metrics-reader
type: kubernetes.io/service-account-token

```

- b. Allow KEDA to authenticate against OpenShift user workload monitoring. To give KEDA the proper permissions, create a **ClusterTriggerAuthentication** CR, which references the **keda-metrics-reader-token** and **Secret** CRs acting as a Token for the **keda-metrics-reader** and **ServiceAccount** CRs created in the previous step. Create the **ClusterTriggerAuthentication** CR:

```

apiVersion: keda.sh/v1alpha1
kind: ClusterTriggerAuthentication
metadata:
  name: ai-inference-keda-thanos
spec:
  secretTargetRef:
    - parameter: bearerToken
      name: keda-metrics-reader-token
      key: token
    - parameter: ca
      name: keda-metrics-reader-token
      key: ca.crt

```

KEDA can now pull metrics from the user workload monitoring stack.

4. Create an inference stack with autoscaling enabled.

- a. Create the namespace.

Because only one inference stack per namespace is supported, create a dedicated namespace for the deployment. You can use any namespace, provided you adjust the manifests accordingly.

```
$ oc create ns autoscaling-example && oc project autoscaling-example
```

b. Create the gateway.

Create a gateway that uses TLS with OpenShift Service Mesh. OpenShift Service Mesh is installed by default on OpenShift version 4.20 and later. To create the **Gateway**, apply the following **ConfigMap** and **Gateway** manifests:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: autoscaling-example-gateway-config
  namespace: autoscaling-example
data:
  service: |
    metadata:
      annotations:
        service.beta.openshift.io/serving-cert-secret-name: "autoscaling-example-gateway-
tls"
    spec:
      type: ClusterIP
  deployment: |
    spec:
      template:
        spec:
          containers:
            - name: istio-proxy
              resources:
                limits:
                  cpu: "16"
                  memory: 16Gi
                requests:
                  cpu: "4"
                  memory: 4Gi
```

+

**NOTE**

This example uses a **ClusterIP** service type for compatibility across OpenShift environments that do not have **LoadBalancer** service type integration. The istio-proxy resource limits are increased to handle the significant load generation required to trigger a scaling event during verification. Adjust these values according to your expected traffic patterns.

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: autoscaling-example-gateway
  namespace: autoscaling-example
spec:
  gatewayClassName: data-science-gateway-class
  infrastructure:
    parametersRef:
      group: ""
      kind: ConfigMap
      name: autoscaling-example-gateway-config
```

```

listeners:
- allowedRoutes:
  namespaces:
    from: Same
  name: https
  port: 443
  protocol: HTTPS
  tls:
    certificateRefs:
    - group: ""
      kind: Secret
      name: autoscaling-example-gateway-tls
    mode: Terminate

```

- c. Create the **LLMInferenceService** CR with autoscaling configurations enabled.

```

apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
  name: autoscaling-example-llama
  namespace: autoscaling-example
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "8000"
    prometheus.io/path: "/metrics"
    security.opendatahub.io/enable-auth: 'false'
spec:
  router:
    scheduler: {}
    route: {}
    gateway:
      refs:
      - name: autoscaling-example-gateway
        namespace: autoscaling-example
  model:
    uri: hf://Qwen/Qwen2.5-7B-Instruct
    name: Qwen/Qwen2.5-7B-Instruct
  labels:
    inference.optimization/acceleratorName: H100
  scaling:
    minReplicas: 1
    maxReplicas: 5
    wva:
      keda:
        pollingInterval: 5
        cooldownPeriod: 30
  template:
    containers:
    - name: main
      image: registry.redhat.io/rhail-early-access/vllm-cuda-rhel9:3.4.0-ea.2
      resources:
        limits:
          cpu: '4'
          memory: 32Gi
          nvidia.com/gpu: 1
        requests:

```

```

cpu: '2'
memory: 16Gi
nvidia.com/gpu: 1
startupProbe:
  httpGet:
    path: /health
    port: 8000
    scheme: HTTPS
readinessProbe:
  httpGet:
    path: /health
    port: 8000
    scheme: HTTPS
livenessProbe:
  httpGet:
    path: /health
    port: 8000
    scheme: HTTPS

```

The following fields are important to understand when you customize the **LLMInferenceService** CR for your use case:

- **.metadata.annotations.security.opendatahub.io/enable-auth:** Disables authentication and rate limiting for this example. In OpenShift AI, user authentication and rate limiting are handled by the Red Hat Connectivity Link Operator. This example disables authentication so that the load test can generate sufficient traffic to trigger a scaling event without being rate-limited. Production deployments should not set this annotation.
- **.spec.gateway.refs:** Specifies the reference to the pre-created **Gateway** that the **LLMInferenceService** uses.
- **.spec.labels.inference.optimization/acceleratorName:** Specifies the type of accelerator this workload uses, such as **A100**, **H100**, or **cpu**. The WVA uses this value to group variants by hardware type and look up accelerator-specific performance data when making scaling decisions. There is no fixed list of valid values: use whatever matches your hardware for semantic identification. If omitted, it defaults to **unknown**.
- **.spec.scaling:** Specifies the user-facing scaling configurations at a per-**LLMInferenceService** level, which represents a per-inference-stack level. Do not confuse this with the configurations from the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap, which represents the default scaling configurations that the **workload-variant-autoscaler** uses to determine the threshold at which it scales your inference workloads up or down.
- **.spec.template.containers[0].image:** Specifies the inference image. Different inference images are used for different accelerators. If you are using accelerators, use the corresponding inference image.
- **.spec.template.containers[0].resources:** Specifies the resource requests and limits. Adjust the resources as needed for more GPUs and add the tensor parallel flags to the **VLLM_ADDITIONAL_ARGS** environment variable, for example **--tensor-parallel 2**.

**NOTE**

The same requirement applies for accelerated networking devices like RoCE or InfiniBand. Autoscaling inter-node workloads through **LeaderWorkerSet** is not supported.

- **.spec.template.containers[0].startupProbe**,
.spec.template.containers[0].readinessProbe,
.spec.template.containers[0].livenessProbe: Specifies the probe configurations. Because this example uses TLS end to end, ensure that your probes use the HTTPS scheme. If the probes do not use the TLS scheme, they fail to reach the endpoints and declare the pods as not ready or not healthy.
- d. Create a **PrometheusRule** to alias inference image metric names. By default, vLLM exposes metrics to the **vllm** namespace, but Red Hat AI Inference Server remaps these metrics to the **kserve_vllm** namespace. The WVA only looks for the **kserve_vllm**-namespaced instances of these metrics. To enable the WVA to pick up these metrics, create the following **PrometheusRule**:

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: vllm-metrics-alias
  namespace: autoscaling-example
  labels:
    monitoring.opendatahub.io/scrape: "true"
spec:
  groups:
    - name: vllm-metric-aliases
      interval: 15s
      rules:
        - record: vllm:kv_cache_usage_perc
          expr: kserve_vllm:kv_cache_usage_perc
        - record: vllm:num_requests_waiting
          expr: kserve_vllm:num_requests_waiting
        - record: vllm:num_requests_running
          expr: kserve_vllm:num_requests_running
        - record: vllm:cache_config_info
          expr: kserve_vllm:cache_config_info
        - record: vllm:request_success_total
          expr: kserve_vllm:request_success_total
        - record: vllm:request_generation_tokens_sum
          expr: kserve_vllm:request_generation_tokens_sum
        - record: vllm:request_generation_tokens_count
          expr: kserve_vllm:request_generation_tokens_count
        - record: vllm:request_prompt_tokens_sum
          expr: kserve_vllm:request_prompt_tokens_sum
        - record: vllm:request_prompt_tokens_count
          expr: kserve_vllm:request_prompt_tokens_count
        - record: vllm:time_to_first_token_seconds_sum
          expr: kserve_vllm:time_to_first_token_seconds_sum
        - record: vllm:time_to_first_token_seconds_count
          expr: kserve_vllm:time_to_first_token_seconds_count
        - record: vllm:time_per_output_token_seconds_sum
          expr: kserve_vllm:time_per_output_token_seconds_sum
```

```
- record: vllm:time_per_output_token_seconds_count
  expr: kserve_vllm:time_per_output_token_seconds_count
- record: vllm:prefix_cache_hits
  expr: kserve_vllm:prefix_cache_hits
- record: vllm:prefix_cache_queries
  expr: kserve_vllm:prefix_cache_queries
```

- e. Verify the **LLMInferenceService** and **ScaledObject** CRs were created successfully.

```
$ oc get llmismvc -n autoscaling-example
```

```
NAME                                URL    READY  REASON  AGE
autoscaling-example-llama          True    102s
```

```
$ oc get scaledobject -n autoscaling-example
```

```
NAME                                SCALETARGETKIND  SCALETARGETNAME
MIN  MAX  READY  ACTIVE  FALLBACK  PAUSED  TRIGGERS
AUTHENTICATIONS  AGE
autoscaling-example-llama-kserve-keda  apps/v1.Deployment  autoscaling-example-
llama-kserve 1  5  True  True  False  False  prometheus  ai-inference-keda-
thanos 119s
```

Verification

Verify autoscaling behavior by confirming that inference server metrics appear in Prometheus, that the WVA emits scaling metrics, and that an end-to-end autoscaling event occurs.

1. Verify that inference server component metrics appear in Prometheus.
Ensure that the inputs to the WVA appear in Prometheus, specifically the inference server metrics:

```
$ TOKEN=$(oc whoami -t)
```

```
$ THANOS=$(oc get route thanos-querier -n openshift-monitoring -o jsonpath='{.spec.host}')
for m in num_requests_running num_requests_waiting kv_cache_usage_perc; do
  curl -sk -G -H "Authorization: Bearer $TOKEN" "https://$THANOS/api/v1/query" \
    --data-urlencode "query=vllm:${m}{namespace=\"autoscaling-example\"}" | \
  jq '.data.result[0]'
done
```

You should see three separate JSON payloads, one for each metric. The following example shows the expected output format:

```
{
  "metric": {
    "__name__": "vllm:num_requests_running",
    "container": "main",
    "endpoint": "8000",
    "engine": "0",
    "instance": "10.129.0.66:8000",
    "job": "autoscaling-example/kserve-llm-isvc-vllm-engine",
    "llm_isvc_component": "workload",
```

```

    "llm_isvc_name": "autoscaling-example-llama",
    "llm_isvc_role": "both",
    "model_name": "Qwen/Qwen2.5-7B-Instruct",
    "namespace": "autoscaling-example",
    "pod": "autoscaling-example-llama-kserve-649dc95869-ntlwq",
    "prometheus": "openshift-user-workload-monitoring/user-workload"
  },
  "value": [
    1774817937.592,
    "0"
  ]
}

```



NOTE

The WVA does not currently use metrics from the scheduler.

2. Verify that the WVA is emitting metrics back to Prometheus.
The metrics to look for are **wva_current_replicas**, **wva_desired_replicas**, and **wva_desired_ratio**. Run the following commands to query these metrics:

```

$ TOKEN=$(oc whoami -t)

$ THANOS=$(oc get route thanos-querier -n openshift-monitoring -o jsonpath='{.spec.host}')
for m in wva_desired_replicas wva_current_replicas wva_desired_ratio; do
  curl -sk -G -H "Authorization: Bearer $TOKEN" "https://$THANOS/api/v1/query" \
    --data-urlencode "query=${m}{exported_namespace=\"autoscaling-example\"}" \
    | jq '.data.result[0]'
done

```

You should see JSON payloads for each metric. The following example shows the expected output format:

```

{
  "metric": {
    "__name__": "wva_desired_replicas",
    "accelerator_type": "H100",
    "endpoint": "https",
    "exported_namespace": "autoscaling-example",
    "instance": "10.128.0.64:8443",
    "job": "workload-variant-autoscaler-controller-manager-metrics-service",
    "namespace": "redhat-ods-applications",
    "pod": "workload-variant-autoscaler-controller-manager-85b8d895cd-x6gnr",
    "prometheus": "openshift-user-workload-monitoring/user-workload",
    "service": "workload-variant-autoscaler-controller-manager-metrics-service",
    "variant_name": "autoscaling-example-llama-kserve-va"
  },
  "value": [
    1774819127.020,
    "1"
  ]
}

```

At this point, verify the following resources that were created from your **LLMInferenceService**:

```
$ oc get va autoscaling-example-llama-kserve-v1 -n autoscaling-example
```

NAME	TARGET	MODEL	OPTIMIZED
METRICSREADY AGE			
autoscaling-example-llama-kserve-v1	autoscaling-example-llama-kserve	Qwen/Qwen2.5-7B-Instruct	1 True 81m

```
$ oc get scaledObject -n autoscaling-example
```

NAME	SCALETARGETKIND	SCALETARGETNAME
MIN MAX READY ACTIVE FALLBACK PAUSED TRIGGERS AUTHENTICATIONS AGE		
autoscaling-example-llama-kserve-keda	apps/v1.Deployment	autoscaling-example-llama-kserve
1 5 True True Unknown False prometheus ai-inference-keda-thanos		
83m		

The **VariantAutoscaling** CR should show **METRICSREADY=True** and the **ScaledObject** should show **READY=true**. Once the WVA controller starts reconciling the **ScaledObject**, it should also become **ACTIVE=True**. If you do not see these values, run **oc describe** on these resources and check their **Status** sections.

3. Verify that the inference worker gets autoscaled.

To make scaling events easier to trigger during testing, set the **kvCacheThreshold** key in the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap in the **redhat-ods-applications** namespace to **0.10**:

```
$ oc edit cm workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications
```

Change **kvCacheThreshold** from **0.80** to **0.10**.

The following script deploys a load-generator pod inside the cluster and watches for scaling events. The script resolves the gateway service, sends thousands of concurrent requests with long token generation to saturate the vLLM KV cache past the WVA scaling threshold, and monitors the replica count every 5 seconds:



NOTE

The scaling behavior and load values in this script are dependent on the accelerators in use. This example was tested with NVIDIA H100 AI accelerators.

```
#!/bin/bash
set -euo pipefail

NS="{1:-autoscaling-example}"
ISVC="{2:-autoscaling-example-llama}"
CONCURRENCY="{3:-200}"
REQUESTS="{4:-5000}"
POD_NAME="load-test-$(date +%s)"
CM_NAME="script-${POD_NAME}"

GATEWAY_SVC=$(oc get svc -n "$NS" -l gateway.networking.k8s.io/gateway-name \
-o jsonpath='{.items[0].metadata.name}' 2>/dev/null)
```



```

if [ -z "$GATEWAY_SVC" ]; then
    echo "ERROR: No gateway service found in namespace $NS"
    exit 1
fi
URL="https://${GATEWAY_SVC}.${NS}.svc.cluster.local/${NS}/${ISVC}/v1/chat/completions"

echo "=== WVA Autoscaling Test ==="
echo "URL:      $URL"
echo "Concurrency: $CONCURRENCY"
echo "Requests:   $REQUESTS"
echo ""
oc get deployment "${ISVC}-kserve" -n "$NS" \
  -o jsonpath='Initial state: {.spec.replicas}/{.status.readyReplicas} ready' && echo ""
echo ""

cleanup() {
    echo ""
    echo "Cleaning up..."
    oc delete pod "$POD_NAME" -n "$NS" --ignore-not-found --wait=false 2>/dev/null
    oc delete configmap "$CM_NAME" -n "$NS" --ignore-not-found 2>/dev/null
}
trap cleanup EXIT INT TERM

read -r -d " LOAD_SCRIPT << 'EOF' || true
#!/bin/sh
set -e
URL="$1"; CONCURRENCY="$2"; REQUESTS="$3"

cat > /tmp/req.sh << 'REQEOF'
#!/bin/sh
curl -sk --max-time 600 "$1" \
  -H "Content-Type: application/json" \
  -d '{"model":"Qwen/Qwen2.5-7B-Instruct","messages":
[{"role":"user","content":"Request $2. Write a detailed essay about topic $2 covering
history, analysis, and predictions."}], "max_tokens":2048}' \
  -o /dev/null -w "req=$2 status=%{http_code} time=%{time_total}s\n"
REQEOF
chmod +x /tmp/req.sh

echo "Smoke test..."
STATUS=$(curl -sk --max-time 30 "$URL" \
  -H "Content-Type: application/json" \
  -d '{"model":"Qwen/Qwen2.5-7B-Instruct","messages":
[{"role":"user","content":"Hi"}], "max_tokens":5}' \
  -o /dev/null -w "%{http_code}")
echo "Status: $STATUS"
if [ "$STATUS" != "200" ]; then
    echo "ERROR: Smoke test failed (HTTP $STATUS)"
    exit 1
fi

echo "Sending $REQUESTS requests ($CONCURRENCY concurrent)..."
seq 1 "$REQUESTS" | xargs -P "$CONCURRENCY" -I{} /tmp/req.sh "$URL" {}
echo "Done."
EOF

```

```

oc create configmap "$CM_NAME" -n "$NS" --from-literal=load.sh="$LOAD_SCRIPT"

cat <<MANIFEST | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: $POD_NAME
  namespace: $NS
spec:
  restartPolicy: Never
  containers:
  - name: load
    image: curlimages/curl
    command: ["sh", "/scripts/load.sh"]
    args: ["$URL", "$CONCURRENCY", "$REQUESTS"]
    resources:
      requests: { cpu: "4", memory: "4Gi" }
      limits: { cpu: "8", memory: "8Gi" }
    volumeMounts:
    - { name: script, mountPath: /scripts }
  volumes:
  - name: script
    configMap: { name: $CM_NAME, defaultMode: 0755 }
MANIFEST

echo "Waiting for pod..."
oc wait --for=condition=Ready pod/"$POD_NAME" -n "$NS" --timeout=120s 2>/dev/null ||
true
sleep 2

POD_PHASE=$(oc get pod "$POD_NAME" -n "$NS" -o jsonpath='{.status.phase}'
2>/dev/null)
if [ "$POD_PHASE" = "Failed" ]; then
  echo "ERROR: Pod failed:"
  oc logs "$POD_NAME" -n "$NS" 2>/dev/null
  exit 1
fi

oc logs -f "$POD_NAME" -n "$NS" 2>/dev/null &
LOGS_PID=$!

echo ""
echo "--- Watching replicas (Ctrl+C to stop) ---"
for _ in $(seq 1 120); do
  replicas=$(oc get deployment "${ISVC}-kserve" -n "$NS" -o jsonpath='{.spec.replicas}'
2>/dev/null)
  ready=$(oc get deployment "${ISVC}-kserve" -n "$NS" -o jsonpath='{.status.readyReplicas}'
2>/dev/null)
  echo " [$(date +%H:%M:%S)] Replicas: ${replicas:-?} (${ready:-0} ready)"
  [ "${replicas:-1}" -gt 1 ] && echo " ** Scale-up detected! **"
  oc get pod "$POD_NAME" -n "$NS" -o jsonpath='{.status.phase}' 2>/dev/null \
    | grep -q "Succeeded|Failed" && echo " Load pod finished." && break
  sleep 5
done

```

```
kill "$LOGS_PID" 2>/dev/null || true
wait "$LOGS_PID" 2>/dev/null || true
```

You should see the replica count increase as load is applied:

```
[17:30:03] Replicas: 1 (1 ready)
[17:30:58] Replicas: 2 (1 ready)
** Scale-up detected! **
```

During the load, you should see additional inference worker pods:

```
$ oc get pods -n autoscaling-example
```

NAME	READY	STATUS	RESTARTS	AGE
autoscaling-example-gateway-data-science-gateway-class-66c4qvpb	1/1	Running	0	81m
autoscaling-example-llama-kserve-56b64d69d-b297x	1/1	Running	0	106m
autoscaling-example-llama-kserve-56b64d69d-kvvmc	1/1	Running	0	10m
autoscaling-example-llama-kserve-56b64d69d-m78nz	1/1	Running	0	7m1s
autoscaling-example-llama-kserve-router-scheduler-77cd5549p4w95	2/2	Running	0	106m

After the load ceases, the deployment scales back down to 1 replica:

NAME	READY	STATUS	RESTARTS	AGE
autoscaling-example-gateway-data-science-gateway-class-66c4qvpb	1/1	Running	0	81m
autoscaling-example-llama-kserve-56b64d69d-b297x	1/1	Running	0	106m
autoscaling-example-llama-kserve-router-scheduler-77cd5549p4w95	2/2	Running	0	106m

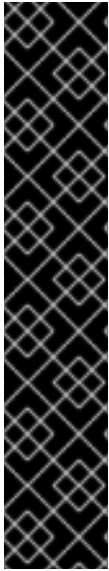


NOTE

The WVA uses a stair-step methodology to scaling, imposing a stabilization window on scaling events so that a traffic burst does not greedily attempt to occupy all your GPUs. If load continues past the stabilization window, you may see additional scaling events.

7.5. VARIANTAUTOSCALING CR CONFIGURATION REFERENCE

You configure autoscaling behavior by creating a **VariantAutoscaling** custom resource (CR).



IMPORTANT

When multiple configuration sources are available, the workload variant autoscaler applies the following precedence order, from highest to lowest.

1. CLI flags
2. Environment variables
3. ConfigMap values
4. Built-in defaults

Namespace-scoped **ConfigMap** resources override cluster-level **ConfigMap** resources for the same parameters. To enable namespace-scoping, add the **wva.llmd.ai/config-enabled: "true"** label to the target namespace.

The following is an example **VariantAutoscaling** custom resource (CR).

```
apiVersion: llmd.ai/v1alpha1
kind: VariantAutoscaling
metadata:
  name: llama-70b-premium-h100
  namespace: llm-inference
  labels:
    app: llm-inference
    model: llama-70b
    variant: premium
    accelerator: h100
spec:
  scaleTargetRef:
    kind: Deployment
    name: llama-70b-premium-h100
  modelID: "meta/llama-3.1-70b"
  minReplicas: 1
  maxReplicas: 5
```

- **metadata.name:** Specifies a name for the **VariantAutoscaling** resource. Use a descriptive name that identifies the model, variant role, and accelerator type.
- **metadata.namespace:** Specifies the namespace where the target deployment runs. Each namespace supports only one Distributed Inference with llm-d inference stack.
- **metadata.labels:** Specifies optional labels for operational tracking. The workload variant autoscaler (WVA) does not use these labels for scaling decisions, but they are useful for filtering and monitoring. The accelerator type is determined by the deployment's actual hardware, not by these labels.
- **spec.scaleTargetRef.kind:** Required. Specifies the kind of the Kubernetes resource that the WVA scales. Supported value: **Deployment**.
- **spec.scaleTargetRef.name:** Specifies the name of the Kubernetes resource that the WVA scales. Must match the name of an existing resource in the same namespace.

- **spec.modelID**: Specifies the OpenAI API-compatible model identifier for the inference workload.
- **spec.minReplicas**: Specifies the minimum number of replicas. The WVA does not scale below this value. Must be **1** or greater. Default: **1**.
- **spec.maxReplicas**: Specifies the maximum number of replicas. The WVA does not scale above this value. Set this value based on your cluster AI accelerator capacity. Default: **2**.

7.6. SATURATION SCALING CONFIGMAP REFERENCE

The workload variant autoscaler (WVA) uses a **ConfigMap** named **workload-variant-autoscaler-saturation-scaling-config** for saturation-based scaling thresholds.

Each **ConfigMap** contains two types of entries:

- A **default** entry that contains global parameters that apply to all models in the scope (cluster-wide or namespace-scoped)
- Optional override entries that contain model-specific customizations with arbitrary names, such as **granite-override** or **llama-override**

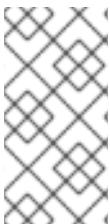
The following example shows a **ConfigMap** CR with cluster-level defaults and a model-specific override:

The **default** entry contains global parameters that apply to all models unless a matching override exists.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: workload-variant-autoscaler-saturation-scaling-config
  namespace: redhat-ods-applications
data:
  default: |
    kvCacheThreshold: <kv_cache_threshold>
    queueLengthThreshold: <queue_length_threshold>
    kvSpareTrigger: <kv_spare_trigger>
    queueSpareTrigger: <queue_spare_trigger>
    enableLimiter: <enable_limiter>
  <override_name>: |
    model_id: "<model_id>"
    namespace: "<namespace>"
    kvCacheThreshold: <kv_cache_threshold>
    kvSpareTrigger: <kv_spare_trigger>
```

- **kvCacheThreshold**: Specifies the KV cache utilization threshold as a percentage. A replica is considered saturated if its KV cache utilization is greater than or equal to this value. Valid range: **0.0** to **1.0**. Lower values trigger scale-up sooner. Default: **0.80**.
- **queueLengthThreshold**: Specifies the queue length threshold. A replica is considered saturated if its queue length is greater than or equal to this value. Lower values trigger scale-up sooner. Default: **5**.

- **kvSpareTrigger**: Specifies the KV cache spare capacity trigger. The WVA signals a scale-up if the average spare KV capacity across non-saturated replicas falls below this value. Valid range: **0.0** to **1.0**. Default: **0.10**.
- **queueSpareTrigger**: Specifies the queue spare capacity trigger. The WVA signals a scale-up if the average spare queue capacity across non-saturated replicas falls below this value. Default: **3**.
- **enableLimiter**: Enables the GPU limiter to constrain scaling based on available cluster GPU capacity. When **true**, the WVA limits desired replicas so that deployments do not request more GPUs than are available in the cluster, preventing new replicas from entering a pending state. Default: **false**.
- **<override_name>**: Override entry name, such as **llm-d** or **granite-override**. The WVA matches overrides by using the **model_id** and **namespace** fields, not the entry name.
- **model_id**: Specifies the model identifier that this override applies to, such as **Qwen/Qwen3-0.6B** or **ibm/granite-13b**. The WVA uses this value together with **namespace** to match the override to a specific model deployment.
- **namespace**: Specifies the namespace of the model deployment that this override applies to. The WVA uses this value together with **model_id** to match the override to a specific model deployment.



NOTE

The WVA resolves configuration by first looking for an override that matches both **model_id** and **namespace**. If no matching override is found, the WVA uses the **default** entry. Any parameters that are not specified in the matched entry receive hardcoded default values.

7.7. AUTOSCALING METRICS

The workload variant autoscaler (WVA) exposes the following autoscaling-specific metrics through Prometheus. These metrics reflect the scaling decisions and state of the WVA controller.

Table 7.1. WVA autoscaling metrics

Metric name	Type	Labels	Description
wva_current_replicas	gauge	variant_name, namespace, accelerator_type	The current number of replicas managed by the WVA for each VariantAutoscaling resource.
wva_desired_replicas	gauge	variant_name, namespace, accelerator_type	The target number of replicas calculated by the WVA. This is the metric read by KEDA or HPA for actuation.

Metric name	Type	Labels	Description
wva_replica_scaling_total	counter	variant_name, namespace, direction, reason	The total number of replica scaling events triggered by the WVA. The direction label is up or down . The reason label indicates the cause of the scaling event.
wva_desired_ratio	gauge	variant_name, namespace, accelerator_type	The ratio of desired replicas to current replicas, indicating the scaling pressure on the deployment.

The WVA uses the following vLLM inference metrics as input signals for scaling decisions. These metrics are exposed by the vLLM model server, not by the WVA controller.

Table 7.2. vLLM saturation metrics used by the WVA

Metric name	Description
kserve_vllm:kv_cache_usage_perc	KV cache utilization as a percentage. A value of 0.0 to 1.0 . The WVA uses max_over_time over a 1-minute window to determine saturation.
kserve_vllm:num_requests_waiting	Number of requests waiting to be processed. The WVA uses max_over_time over a 1-minute window to determine queue depth.



NOTE

The vLLM saturation metrics must include **pod**, **model_name**, and **namespace** labels for the WVA to correlate metrics with the correct **VariantAutoscaling** resource.

7.8. HPA BEHAVIOR DEFAULTS

The workload variant autoscaler (WVA) uses KEDA to manage HPA resources. KEDA configures the following default HPA scaling behavior. These values control the stabilization window and scaling velocity.

Table 7.3. HPA scaling behavior defaults

Parameter	Default	Description
scaleUp.stabilizationWindow Seconds	240	The stabilization window for scale-up decisions, in seconds. The HPA waits this long after the last scaling event before scaling up again.

Parameter	Default	Description
scaleDown.stabilizationWindowSeconds	240	The stabilization window for scale-down decisions, in seconds.
selectPolicy	Max	The policy selection strategy. Max selects the policy that results in the largest change.
policies[0].type	Pods	The scaling policy type.
policies[0].value	10	The maximum number of pods to add or remove per scaling period.
policies[0].periodSeconds	150	The duration of the scaling period, in seconds.

7.9. TROUBLESHOOTING THE WORKLOAD VARIANT AUTOSCALER

If your workload variant autoscaler (WVA) deployment is not scaling as expected, use the following troubleshooting steps to diagnose and resolve common issues.

- Check the WVA controller status:
View WVA controller logs to diagnose issues:

```
$ oc logs -l app.kubernetes.io/name=workload-variant-autoscaler -n redhat-ods-applications
```

- Verify **VariantAutoscaling** resource conditions:
Check the status conditions on your **VariantAutoscaling** resources:

```
$ oc describe va <variant_name> -n <namespace>
```

Common condition states:

- **MetricsAvailable: True** with reason **MetricsFound**: Prometheus metrics are available
- **MetricsAvailable: False** with reason **MetricsMissing**: No metrics found for the variant. Check that vLLM metrics are being scraped.
- **MetricsAvailable: False** with reason **PrometheusError**: WVA cannot connect to Prometheus. Check Prometheus configuration and network connectivity.
- **OptimizationReady: True** with reason **OptimizationSucceeded**: WVA successfully calculated scaling decisions
- **OptimizationReady: False** with reason **OptimizationFailed**: Scaling calculation failed. Check WVA logs for details.
- Verify that KEDA resources are functioning correctly:
Check the status of the KEDA **ScaledObject** resources in your namespace:

```
$ oc get scaledobjects -n <namespace>
```


Check the KEDA operator logs for errors:

```
$ oc logs -l app=keda-operator -n openshift-keda
```

- Verify Prometheus metrics are being collected:
Confirm that vLLM metrics are available in Prometheus:

```
$ oc port-forward -n openshift-monitoring prometheus-k8s-0 9090:9090
```

Then query Prometheus at **http://localhost:9090** for **vllm:kv_cache_usage_perc** and **vllm:num_requests_waiting**.

- Check the WVA scaling metrics in Prometheus:
Query the **wva_desired_replicas** metric to verify the WVA is calculating scaling targets:

```
$ oc exec -n openshift-monitoring prometheus-k8s-0 -- promtool query instant  
http://localhost:9090 'wva_desired_replicas'
```

- Enable user workload monitoring:
If metrics are not being collected, verify that OpenShift User Workload Monitoring is enabled:

```
$ oc get configmap cluster-monitoring-config -n openshift-monitoring
```

If the ConfigMap does not exist, create it:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

CHAPTER 8. MANAGING MIXED WORKLOADS WITH PRIORITY QUEUING

With flow control, you can offer different quality of service for latency-sensitive and throughput-sensitive workloads, or clients, on the same model servers without violating service-level agreements (SLAs), reducing capital expenses and maximizing accelerators return on investment.



IMPORTANT

flow control is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

8.1. FLOW CONTROL AND PRIORITY-BASED QUEUING

Flow control is a pool defense mechanism in the Endpoint Picker (EPP) component of Distributed Inference with llm-d. It manages request queuing, prioritization, and fairness in a multi-tenant inference serving environment, which helps with consolidating latency-sensitive and throughput-sensitive workloads on a single cluster, reducing infrastructure costs while protecting critical service level objectives (SLOs).

Use flow control in the following scenarios:

- You run many tenant applications with different service level agreement (SLA) requirements on shared GPU infrastructure
- You need to guarantee latency for interactive workloads while maximizing GPU usage with batch jobs
- You want to merge separate Distributed Inference with llm-d deployments for different service tiers onto a single cluster
- You need to protect critical workloads from noisy neighbor effects
- You want to implement service tiering for multi-tenant LLM inference

If you only have a single workload type with uniform SLA requirements, flow control adds unnecessary complexity.

How Flow Control works

Flow control manages request queuing, prioritization, and fairness between the Inference Gateway and InferencePool backends.

When a request arrives:

1. The Gateway's **AuthPolicy** automatically injects two headers based on authentication:
 - **x-gateway-inference-fairness-id** - Groups requests from the same authentication source for fair queuing

- **x-gateway-inference-objective** - Determines request priority by matching an **InferenceObjective** resource
2. The request enters a priority-aware queue in the Endpoint Picker and waits for dispatch.
 3. The dispatch logic applies strict priority ordering:
 - Higher-priority requests dispatch before lower-priority requests
 - Requests at the same priority level use fairness policies. The default **global-strict-fairness-policy** is a greedy strategy that ignores tenant boundaries and offers no isolation. A single tenant in a priority can starve other tenants of the same priority. For equitable service distribution, configure **round-robin-fairness-policy**
 4. A saturation check monitors queue depth and KV cache utilization on model servers:
 - When pool saturation is detected (saturation ≥ 1), dispatch is halted for all priority bands
 - All requests wait in their respective priority queues until capacity becomes available
 5. When capacity is available, the request proceeds to the Scheduling layer and routes to a specific model server.

Flow control queues requests when the **InferencePool** has no ready backends, hiding startup time from clients during scale-from-zero scenarios.

Key concepts

Priority levels

Platform administrators create **InferenceObjective** resources to define priority tiers. Higher integer values represent higher priority.

Example tier structure:

Tier	Priority Value	Use Case
Critical	100	Interactive applications
Standard	0 (default)	Standard API calls
Best-Effort	-10	Batch workloads

Authentication-based prioritization

The Gateway **AuthPolicy** automatically sets the **x-gateway-inference-objective** header:

- ServiceAccount tokens: Header is set to the ServiceAccount's namespace
 - User tokens: Header is set to **authenticated**
 - Anonymous requests: Header is set to **unauthenticated**
- Create **InferenceObjective** resources with names matching these header values to assign priorities.

8.2. INFERENCEOBJECTIVE CUSTOM RESOURCE REFERENCE

The **InferenceObjective** Custom Resource defines priority values for inference requests in flow control. You create **InferenceObjective** resources to establish priority tiers for different workload classes.

```
apiVersion: inference.networking.x-k8s.io/v1alpha2
kind: InferenceObjective
metadata:
  name: <service-account-namespace>
  namespace: <llmisvc-namespace>
spec:
  priority: 100 # Higher = higher priority
  poolRef:
    group: inference.networking.k8s.io
    kind: InferencePool
    name: <llmisvc-name>-inference-pool
```

Table 8.1. InferenceObjective specifications

Field	Type	Required	Description	Valid Values
priority	integer	No	Priority value for requests matching this InferenceObjective . Higher integer values represent higher priority. If not specified, defaults to 0.	Positive values (higher priority, example: 100), Zero (default priority), Negative values (lower priority, dispatched last, example: -10)
poolRef.group	string	No	API group for the InferencePool resource. If not specified, defaults to inference.networking.k8s.io .	inference.networking.k8s.io
poolRef.kind	string	No	Resource kind. If not specified, defaults to InferencePool .	InferencePool
poolRef.name	string	Yes	Name of the InferencePool resource this InferenceObjective applies to.	Name of your InferencePool resource

8.3. CONFIGURE FLOW CONTROL FOR DISTRIBUTED INFERENCE WITH LLM-D

You can enable flow control in Distributed Inference with llm-d and create **InferenceObjective** resources to implement priority-based queuing for multitenant workloads. This procedure configures the flow control layer, defines priority tiers, and validates that priority queuing is working correctly.

Prerequisites

- Distributed Inference with llm-d is deployed with Inference Gateway and Endpoint Picker (EPP).
- You have enabled authentication and authorization for your LLM Inference Service. To do this, follow the steps for [Enabling authentication and authorization for an LLM inference service](#)
- You have cluster administrator access.

Procedure

1. Configure the Endpoint Picker to enable flow control and configure saturation detection. The **EndpointPickerConfig** is embedded within the **LLMInferenceService** custom resource under **spec.router.scheduler**. Edit your **LLMInferenceService** resource:

```
$ oc edit llminferenceservice <llm-service-name> -n <namespace>
```

2. Locate the **spec.router.scheduler** section and add the **flowControl** feature gate and configure the **saturationDetector**:

```
apiVersion: inference.networking.x-k8s.io/v1alpha1
kind: EndpointPickerConfig
featureGates:
  - "flowControl"
saturationDetector:
  queueDepthThreshold: 5
  kvCacheUtilThreshold: 0.8
  metricsStalenessThreshold: 200ms
```

where:

- **queueDepthThreshold**: Specifies the queue depth threshold on model servers. A value of 5 (default) balances latency and throughput. Set to 1 for maximum fairness (forces all queuing into EPP priority-aware queues). Set to 1-2x max batch size for maximum throughput (allows model servers to form fuller batches). To decide your model server's batch size for tuning **queueDepthThreshold**, check the vLLM deployment's **--max-batch-total-tokens** parameter or refer to your **LLMInferenceService** configuration. If unsure, start with the default value of 5 and adjust based on observed latency and throughput metrics.
 - **kvCacheUtilThreshold**: Specifies the KV cache utilization threshold (0.0 to 1.0). Default is 0.8.
 - **metricsStalenessThreshold**: Specifies how long metrics can be stale before being considered invalid. Default is 200ms.
3. Save and exit the editor.

4. Create an **InferenceObjective** resource to define a priority tier:

```
apiVersion: inference.networking.x-k8s.io/v1alpha2
kind: InferenceObjective
metadata:
  name: <objective-name>
  namespace: <namespace>
spec:
  priority: <priority-value>
  poolRef:
    group: inference.networking.k8s.io
    kind: InferencePool
    name: <inference-pool-name>
```

where:

- **metadata.name** - Specifies the **InferenceObjective** name.
- **metadata.namespace** - Specifies the namespace where the **InferencePool** is deployed.
- **priority** - Specifies the priority value. Higher integer values represent higher priority. Negative values designate lower-priority requests that are queued and dispatched last. Requests can be rejected when:
 - The priority band's capacity is exhausted (HTTP 429)
 - They time out (HTTP 503)
 - The client disconnects abruptly (HTTP 503)
- **poolRef.name** - Specifies the name of the **InferencePool** resource this objective applies to.

5. Apply the **InferenceObjective** resource:

```
$ oc apply -f <inference-objective-file>.yaml
```

6. Optional: Configure advanced flow control settings for priority bands.

After creating **InferenceObjective** resources, you can customize flow control behavior by configuring priority band capacity, request timeouts, and fairness policies. Edit your **LLMInferenceService** resource and add the **flowControl** configuration under **spec.router.scheduler**:

```
apiVersion: inference.networking.x-k8s.io/v1alpha1
kind: EndpointPickerConfig
featureGates:
  - "flowControl"
flowControl:
  maxBytes: 1073741824
  defaultRequestTTL: 1m
  priorityBands:
    - priority: 1
      orderingPolicyRef: fcfs-ordering-policy
      fairnessPolicyRef: round-robin-fairness-policy
    - priority: 0
```

```

orderingPolicyRef: fcfs-ordering-policy
fairnessPolicyRef: round-robin-fairness-policy
- priority: -1
  orderingPolicyRef: fcfs-ordering-policy
  fairnessPolicyRef: round-robin-fairness-policy

```

where:

- **maxBytes** - Specifies the priority band's capacity in bytes. When this limit is exceeded, requests receive HTTP 429 errors. Default is 1073741824 (1 GiB).
- **defaultRequestTTL** - Specifies the request timeout. Requests waiting longer than this duration receive HTTP 503 errors. Default is 1 minute.
- **priorityBands[].priority** - Specifies the priority value for this band. Must match the priority values defined in your **InferenceObjective** resources.
- **priorityBands[].orderingPolicyRef** - Specifies the ordering policy. Default is **fcfs-ordering-policy** (first-come, first-served).
- **priorityBands[].fairnessPolicyRef** - Specifies the fairness policy. The default **global-strict-fairness-policy** offers no tenant isolation and can result in starvation. For equitable service distribution across tenants, use **round-robin-fairness-policy**.

Verification

1. Send sustained traffic with varying request priorities using load testing tools.
Priority differentiation is active whenever there are multiple requests waiting in the queue. The saturation signal controls whether anything gets dispatched at all. Use load testing tools to send sustained traffic with varying request priorities, tenant fairness IDs, and concurrent request volumes.
2. Monitor flow control metrics in Prometheus:
 - **inference_extension_flow_control_queue_size** - Queue depth for requests waiting to dispatch
 - **inference_extension_flow_control_request_queue_duration_seconds** - Time requests sit in the flow control queue
3. If queue depth remains at 0, increase concurrency to create saturation conditions where priority differentiation occurs.

Additional resources

- [Configure scheduler for LLM inference services](#)
- [Flow Control metrics](#)

Additional resources

- [Configure scheduler for LLM inference services](#)
- [Flow Control metrics \(upstream\)](#)

